

Bryce Han — bhan16@ucsc.edu — Github: “BHan2002”

Shawn Kramer — skkramer@ucsc.edu — Github: “Shawn-Kramer”

Ivan Li — ivli18@ucsc.edu — “ivli18”

Custom Shaders Showcase

Water Shader:

- `_WaterColor`: the base color of the water surface
- `_FoamColor`: the color that appears at glancing angles via the fresnel effect
- `_WaveSpeed`: how fast the vertex displacement animates over time
- `_WaveHeight`: the amplitude of the wave displacement
- `_WaveFrequency`: how tightly packed the waves are
- `_FresnelPower`: controls the sharpness of the edge transparency/foam effect
- `_NormalMap`: tileable water normal texture that adds surface ripple detail
- `_NormalScrollSpeed`: the speed and direction the two normal map layers scroll

```
Varyings vert(Attributes v)
{
    Varyings o;

    // Two waves offset in frequency and phase
    float wave1 = sin(v.positionOS.x * _WaveFrequency + _Time.y * _WaveSpeed) * _WaveHeight;
    float wave2 = sin(v.positionOS.z * _WaveFrequency * 0.8 + _Time.y * _WaveSpeed * 1.3 + 1.5) * _WaveHeight * 0.6;
    v.positionOS.y += wave1 + wave2;

    // Transform to clip space
    o.positionHCS = TransformObjectToHClip(v.positionOS.xyz);
    // UVs for normal map scrolling
    o.uv = TRANSFORM_TEX(v.uv, _NormalMap);
    // Transform normal to world space for lighting calculations
    o.normalWS = TransformObjectToWorldNormal(v.normalOS);
    o.viewDirWS = normalize(GetWorldSpaceViewDir(TransformObjectToWorld(v.positionOS.xyz)));
    return o;
}

half4 frag(Varyings i) : SV_Target
{
    // Scroll two layers of normals in different directions for more dynamic waves
    float2 uv1 = i.uv + _Time.y * _NormalScrollSpeed.xy;
    float2 uv2 = i.uv + _Time.y * _NormalScrollSpeed.xy * float2(-0.7, 1.2) + float2(0.4, 0.2);

    // Sample and blend normals
    float3 n1 = UnpackNormal(SAMPLE_TEXTURE2D(_NormalMap, sampler_NormalMap, uv1));
    float3 n2 = UnpackNormal(SAMPLE_TEXTURE2D(_NormalMap, sampler_NormalMap, uv2));
    // Blend the two normal samples together
    float3 blendedNormal = normalize(n1 + n2);

    // Fresnel
    float fresnel = pow(1.0 - saturate(dot(blendedNormal, normalize(i.viewDirWS))), _FresnelPower);

    // Blend between water color and foam color based on Fresnel term
    half4 col = lerp(_WaterColor, _FoamColor, fresnel);
    // Make foam more opaque at glancing angles
    col.a = lerp(_WaterColor.a, 1.0, fresnel * 0.5);
    return col;
}
```

Dissolve Shader:

- `_BaseMap`: the albedo texture of the object being dissolved
- `_BaseColor`: a tint multiplied over the base texture
- `_DissolveMap`: grayscale noise texture that determines which pixels dissolve first
- `_DissolveAmount`: manual 0–1 control over how far the dissolution has progressed
- `_DissolveSpeed`: how fast the dissolution loops
- `_AutoDissolve`: toggles between manual slider (0) and automatic time-based animation (1)
- `_EdgeColor`: the emission color of the glowing rim at the dissolve boundary
- `_EdgeWidth`: thickness of the glowing edge band
- `_EdgeIntensity`: brightness multiplier for the edge glow

```
half4 frag(Varyings IN) : SV_Target
{
    // Sample textures
    half4 baseColor = SAMPLE_TEXTURE2D(_BaseMap, sampler_BaseMap, IN.uvBase) * _BaseColor;
    float noise      = SAMPLE_TEXTURE2D(_DissolveMap, sampler_DissolveMap, IN.uvDissolve).r;

    // Compute threshold – loops 0+1 automatically, or use manual slider
    float autoThreshold = frac(_Time.y * _DissolveSpeed);
    float threshold     = lerp(_DissolveAmount, autoThreshold, _AutoDissolve);

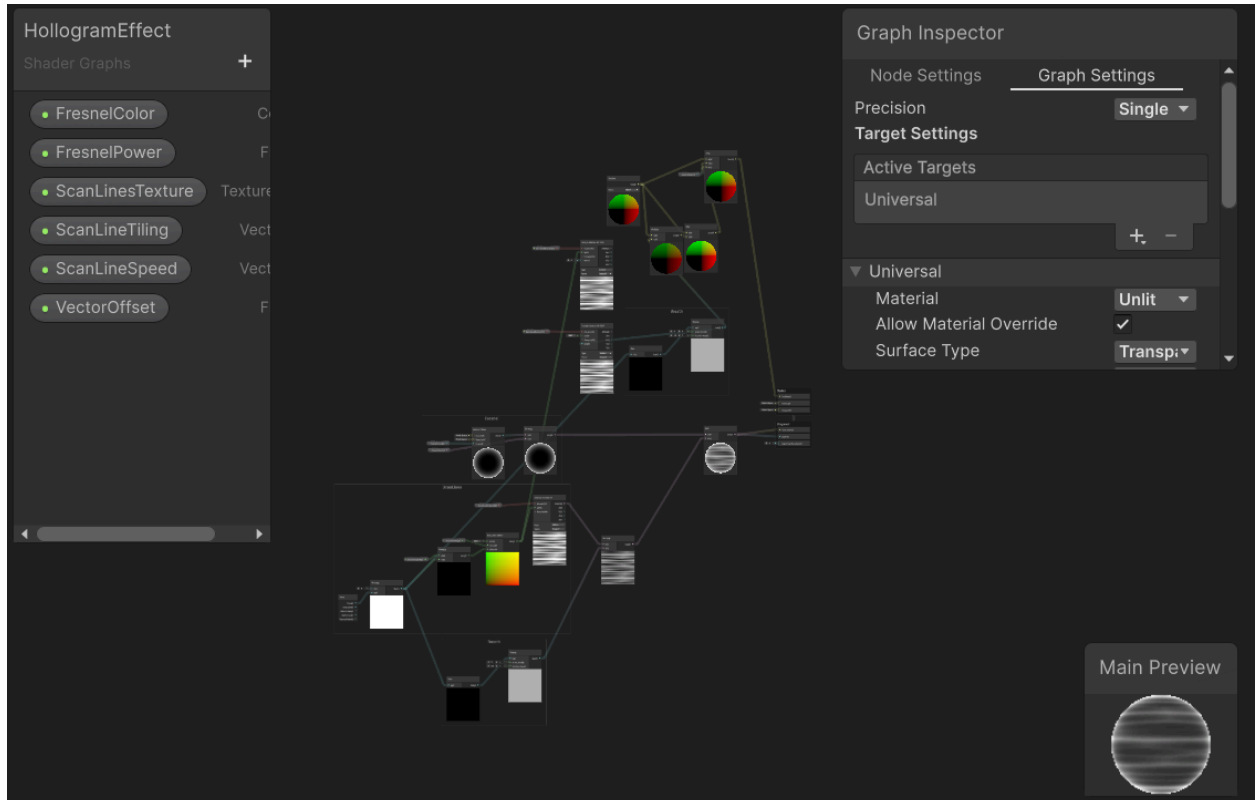
    // Discard this pixel if noise is below threshold
    clip(noise - threshold);

    // Glow edge – pixels just above the threshold get colored
    float edgeMask = 1.0 - smoothstep(0.0, _EdgeWidth, noise - threshold);
    float3 glow    = _EdgeColor.rgb * edgeMask * _EdgeIntensity;

    return half4(baseColor.rgb + glow, 1.0);
}
```

Hologram Shader (Graph):

- FresnelColor: the HDR color of the rim glow around the object edges
- FresnelPower: sharpness of the fresnel rim effect
- ScanLinesTexture: repeating texture used for the horizontal scan line pattern
- ScanLineTiling: how densely the scan lines tile across the surface
- ScanLineSpeed: direction and speed the scan lines scroll over time
- VectorOffset: offsets the sine wave used for the hologram breathing/pulse animation



Reflections:

Bryce Han:

For this assignment, I worked on creating a shader using Shader Graph, focusing on building the visual effect through node-based logic rather than writing code directly. Seeing the live previews inside each node helped a lot, since I could immediately tell how changes affected the output instead of guessing what the math was doing. One of the biggest things I learned from this process was how visual effects are often built from smaller, simpler operations layered together. Working with Shader Graph also helped me better understand how data flows through a system and how important it is to organize the graph clearly.

Shawn Kramer:

My contribution was everything related to the dissolve shader. I learned about the ingredients needed to make this shader, including writing the script, adding a noise map, and the various settings to make it look better, such as edge glow, edge width, auto animate, and auto dissolve speed. A lot of it was not as hard as I thought it would be, and once I got started I managed to figure everything out.

Ivan Li:

For this assignment I implemented the water surface shader, as well as the minimal camera movement to view the scene at differing angles. One thing I learned was how the fresnel effect works, taking the dot product of the surface's normal against the view direction in order to make the structure behave/appear differently based on the viewing angle. Another thing I learned was about how UV manipulation works for scrolling two copies of a normal map, and the actual application of it in practice. On top of that, the other features of the shader helped to make it feel more alive, such as the varying frequencies and heights for the waves. That being editable also makes it such that the details of the shader were easier to tweak than I would've originally thought.